

WEEKLY INTERNSHIP REPORT

SOFTWARE DEVELOPMENT ENGINEERING — WEEK 1

1. Document & Internship Information

Intern Name	Jil Patel
Roll Number	24BCP098
University	Pandit Deendayal Energy University
Project Title	User Management System with Secure Authentication
Tech Stack	ASP.NET Core MVC (C#), PostgreSQL, Entity Framework Core, Bootstrap 5, JWT
Company Name	ncode Solutions (GNFC)
Company Address	(n) Code Solutions (A Division of GNFC Ltd.), 14th Floor, Tower One, Road 5C, Zone 5, Gujarat International Finance Tec-City, Gandhinagar, Gujarat 382050
Internship Role	Software Developer Intern
Work Mode	On-Site
Reporting Manager	Hemal Shah (Manager)
Duration Covered	18 May 2026 – 24 May 2026

2. Project Overview & Objectives

The core objective during this primary week of my internship at ncode Solutions was to initiate, architect, and deliver a production-ready **User Management System**. This system serves as an enterprise foundation for secure identity management, requiring a hybrid authentication architecture: stateful browser sessions to govern the standard web UI and stateless, cryptographically signed tokens to secure modular API endpoints.

Key Structural Objectives:

- Build a scalable MVC folder architecture decoupling business logic from standard data-access layers.
- Model a secure database relational structure mapping users, roles, and privileges using **PostgreSQL**.
- Enforce strict server-side model validations and integrity constraints to minimize security risk vectors.

- Deploy robust identity protection features, including cryptographic password hashing and token-gated endpoint access via **JSON Web Tokens (JWT)**.

3. Detailed Day-wise Work Description

Day 1: Project Setup, Dependency Injections, & Core Middleware Configuration

Initialized the baseline web application layer utilizing Visual Studio scaffolding templates. Built a strict Model-View-Controller (MVC) directory structure to decouple data objects, processing operations, and client layouts.

- Configured the native dependency injection container within `Program.cs` to manage web services lifecycles.
- Installed core data management drivers using the NuGet package ecosystem, explicitly importing `Microsoft.EntityFrameworkCore`, `Microsoft.EntityFrameworkCore.Design`, and `Npgsql.EntityFrameworkCore.PostgreSQL`.
- Formulated global configurations inside `appsettings.json`, explicitly declaring the physical database connection strings to establish a runtime pipeline with the local PostgreSQL environment.

Day 2: Relational Database Modeling, Schema Mapping, & EF Core Migrations

Designed the persistent storage layout by crafting specialized entity representations using strongly typed C# classes.

- Authored a foundational `User` identity model tracking parameters including: *FullName*, *Username*, *Email*, *Mobile Number*, *Password*, *Date of Birth*, and *RoleId*.
- Designed a relational layout creating a separate `RoleMaster` table (storing Admin and User profiles) linked dynamically via foreign keys to the `UserMaster` data table to enforce role boundaries directly at the schema layer.
- Declared an active communication context class inheriting from `DbContext`, mapping collections to entities using `DbSet<User>` arrays.
- Executed database migration operations via the Package Manager Console (Add-Migration `InitialCreate` followed by `Update-Database`), converting abstract definitions into operational physical tables inside PostgreSQL.

Day 3: Secured Registration Workflow & Server-Side Input Auditing

Engineered an onboarding system capable of screening input data integrity and blocking invalid entries prior to database ingestion.

- Constructed declarative validation schemes utilizing `DataAnnotations` properties directly on registration view models (such as `[Required]`, `[EmailAddress]`, and `[StringLength]`).
- Wrote server-side checking logic ensuring strict confirmation string parity (matching Password and Confirm Password inputs).
- Enforced operational business boundaries by implementing constraints that explicitly reject registrations registering a future date as the user's Date of Birth.
- Mitigated data deduplication risks by deploying asynchronous LINQ checks (`.AnyAsync()`) to verify that incoming Email records and Mobile Numbers are entirely unique.
- Prevented plain-text credential vulnerabilities by passing ingestion logic through ASP.NET Core's cryptographically secure `PasswordHasher<User>` engine.

Day 4: State Maintenance, Cryptographic Verification, & Session Lifecycles

Engineered login handling logic to verify credentials against database records while establishing web state mechanisms.

- Formulated secure lookup queries verifying usernames against database records; if identified, the cryptographic hash was evaluated using the `PasswordVerificationResult` utility.
- Activated cross-request state tracking across the web controllers by registering and structuring the session middleware pipeline inside `Program.cs` via `builder.Services.AddSession()`.
- Upon successful validation, user profile states were cached directly into memory using `HttpContext.Session.SetString()` and `SetInt32()`, capturing the active client's Username, UserId, and unique RoleId.
- Implemented an automated context interceptor that audits session variables to route successful log-ins to their respective landing dashboards.

Day 5: Multi-Tier Metrics Dashboard & Privilege-Bound CRUD Controls

Constructed an administrative management interface dashboard and applied precise role restrictions to separate data access routes.

- Programmed a metrics dashboard backend running optimized aggregate queries to compile real-time workspace counters, including Total System Users, Total Admins, and Regular User distributions.
- Built a secure profile layout pulling identifiers directly from active session parameters to safely present user-specific records.
- Locked down administrative controls by placing rigorous access checks over data channels; regular users are systematically blocked, while validated admins gain access to view full user lists, alter account properties, and trigger deletion routines.

Day 6: Responsive Layout Refactoring, Component Styling, & Feedback Loops

Upgraded front-end layouts into an adaptive corporate interface using Bootstrap 5 structural components.

- Implemented a dual-axis dashboard structure comprising a collapsible sidebar navigation module and a top utility contextual navbar displaying the active user's name.
- Formatted unorganized data lists into responsive data tables complete with color-coded contextual role badges for cleaner scanning.
- Deployed a professional visual feedback loop using temporary state data bags (TempData), enabling the layout wrapper to automatically trigger clean, animated alert notifications on success or error events.

Day 7: Stateless JSON Web Token (JWT) Core & Postman API Evaluation

Expanded system capabilities by architecting a programmatic REST API layer running in parallel with traditional web view controllers.

- Programmed a secure token assembly routine using structural encryption keys, generating signed JSON Web Tokens (JWT) using the high-entropy `SecurityAlgorithms.HmacSha256` configuration.
- Bundled identity claims—such as the unique Username, localized User ID, and specific security roles—directly inside the cryptographic token payloads.
- Created separate api-bound controllers providing standardized JSON responses for registration and authentication transactions.
- Protected sensitive data endpoints via the `[Authorize]` filter, requiring incoming calls to present a verified, unexpired Bearer token inside the HTTP header.
- Audited endpoint behaviors comprehensively using Postman, running explicit request structures to verify correct 200 OK responses, 401 Unauthorized exceptions, and 403 Forbidden boundaries across all user profiles.

4. Task Status Summary

#	TASK DESCRIPTION	CATEGORY	STATUS
1	MVC Architectural Directory Setup & Dependency Mapping	Backend	COMPLETED
2	PostgreSQL Provider Setup & Connection String Setup	Database	COMPLETED
3	DB Data Context Blueprinting & Data Table Relationships	Database	COMPLETED
4	Entity Property Modeling & Database Schema Migrations	Database	COMPLETED
5	Data Annotation Constraints & Server Input Validation	Backend	COMPLETED
6	PasswordHasher Cryptographic Integration	Security	COMPLETED
7	Credential Auditing Logic & Onboarding Pipelines	Backend	COMPLETED
8	Session Middleware Pipeline Activation (Program.cs)	Security	COMPLETED
9	Stateful Property Ingestion (HttpContext.Session)	Security	COMPLETED
10	Metrics Aggregator Queries & Analytics Dashboard Count Cards	Backend	COMPLETED
11	Administrative Controller Boundaries & Action Filters	Security	COMPLETED
12	Core UI Components Layout (Sidebar, Profile View, Tables)	Frontend	COMPLETED
13	Bootstrap 5 Layout Refactoring & TempData Alerts	UI/UX	COMPLETED
14	Symmetric Security Key Generation & JWT Claims Factory	Security	COMPLETED
15	Token-Gated API Controllers with [Authorize]	Security	COMPLETED
16	End-to-End Route Validation & Payload Testing via Postman	Testing	COMPLETED

5. Challenges Faced & Solutions Implemented

Challenge 1: Dual-State Hybrid Authentication Conflicts

Coordinating stateful cookie tracking alongside stateless tokenized HTTP Authorization headers caused internal middleware parsing issues, occasionally leading to unintended cross-routing loops or unprompted authorization requests on API endpoints.

Solution:

Explicitly separated the authentication configuration layers within `Program.cs`. Configured independent scheme routers, establishing localized Cookie-based lifecycles strictly for standard MVC UI Controllers, while binding independent JWT Bearer parsing engines exclusively to api-bound endpoints.

Challenge 2: Time-Zone Type Faults with PostgreSQL DateTime Schema

PostgreSQL rejected incoming database mutations with clear exceptions when attempting to persist native C# `DateTime` instances, due to implicit formatting differences regarding localized time offset stamps.

Solution:

Normalized all model-tier timestamp creation parameters. Configured application logic to systematically convert values into Universal Coordinated Time formatting (`DateTime.UtcNow`) prior to tracking mutations or updating records in the database.

Challenge 3: Route Binding Collisions and HTTP Error Code Traps

Overlapping endpoint route definitions and missing route constraints initially threw frequent 400 Bad Request, 404 Not Found, and 405 Method Not Allowed errors during early Postman API testing cycles.

Solution:

Refactored controller routing configurations by deploying explicit routing tokens (`[Route("api/[controller]")]`) and mapping distinct HTTP verb decorators (`[HttpPost]`, `[HttpGet]`) over all endpoint systems to prevent method collisions.

6. Key Learning Outcomes

- **Enterprise Framework Architecture:** Gained deep practical understanding of the request lifecycle, dependency container registrations, and middleware pipelines within ASP.NET Core MVC.

- **Relational Database Management:** Mastered the management of structural tables, relational mapping constraints, and schema migration rollouts via Entity Framework Core connected to PostgreSQL.
- **Hybrid Security Foundations:** Acquired technical proficiency in differentiating and implementing stateful server-side Sessions and stateless cryptographic identity tokens (JWT) based on production architectural constraints.
- **Production Diagnostics & Analysis:** Gained proficiency in tracking network headers, analyzing request/response structures, and debugging runtime HTTP error states utilizing Postman.

7. Conclusion

The first week of my internship at ncode Solutions (GNFC) achieved its structural objectives: designing and executing a fully operational, high-security foundation for the User Management System. Tackling full-stack challenges across relational schema creation, backend password hashing, and token-secured API routing provided excellent hands-on software development exposure matching modern industry production standards.

Submitted By:

Jil Patel
Software Developer Intern
Pandit Deendayal Energy University